420-SN1-RE: Programming In Science Fall 2025

Assignment 2 - Saving baby turtles

1. 📝 **Worth:** 6%
2. 🗓️ **Due:** Posted on LEA
3. 🕐 **Late submissions:** a penalty of 10% per day late will be deducted.
4. 📩 **Submission:**
   - Submit ONLY `turtle-projectile.py`
   - ⚠️ OR: if you have extra `.gif` files, put them and `turtle-projectile.py` in a `.zip` folder and submit that instead.

Premise

Using the Turtle library, your task in this assignment will be to create a turtle-saving simulation.

The user of your simulation must try to throw a stranded turtle into the ocean – saving it from being stranded. The user will be able to control the turtle's **angle** and **initial velocity**.

Your job, as the programmer, is to chart the path of the turtle, and detect if the user's throw was accurate: either saving the turtle, or something else (I'm sure the turtle will be fine).

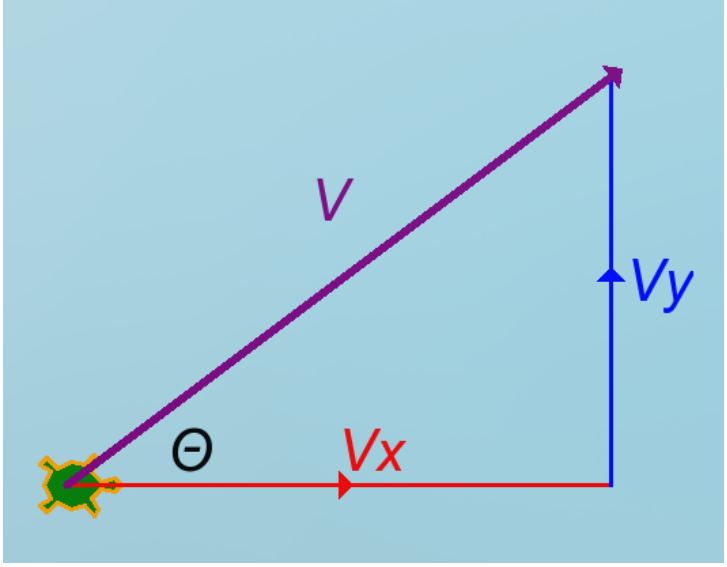
Theory

In order to simulate the flight path of the turtles we are rescuing, we need to be able to **model** the motion of a thrown turtle. Let's start with some definitions:

- Let (x, y) be the position of the turtle at a given time
- Let $\vec{v}$ be the velocity vector for the turtle, which has a magnitude $v = |\vec{v}|$ and an

angle θ

- Let (v_x, v_y) be the x and y components of $\vec{v}$
- Let g represent gravitational acceleration
- Let Δt be some amount of time that has elapsed



The magnitude of the velocity vector |V| and the angle of the trajectory Θ are given to you -- you can calculate the individual components Vx and Vy using trigonometric functions.

Given the above concepts, the motion of a turtle should follow the equations of motion:

$$(1) \quad x \leftarrow x + v_x \cdot \Delta t$$

$$(2) \quad y \leftarrow y + v_y \cdot \Delta t - (1/2)g \cdot \Delta t^2$$

$$(3) \quad v_x \leftarrow v_x$$

$$(4) \quad v_y \leftarrow v_y - g \cdot \Delta t$$

That is:

- The turtle's position (x, y) is affected by the turtle's velocity in each direction over the time elapsed
- The turtle's position is also affected by the integration of acceleration due to gravity over the same time elapsed
- The turtle's velocity in the x direction never changes (assume no friction)
- The turtle's velocity in the y direction is affected by gravitational acceleration over the time elapsed

Your job will be to use Python to track the turtle's position over time, and determine whether the turtle lands in safety or danger.

Tasks

To create the simulation, complete the tasks below.

0. Getting started

Like all labs and assignments, you'll need to:

- Create a new Pycharm project in your SN1 OneDrive folder
- Create a python file inside that project (name it `turtle-projectile.py`)

All of your python code can go in `turtle-projectile.py` . Some code you'll find useful to start with:

```
# These lines of code should go at the top of your program
import math
import turtle
```

Some helpful turtle defaults you can set:

```
turtle.speed(0)  # this is the fastest speed
turtle.hideturtle() # this hides the default turtle
turtle.bgcolor("lightblue") # this sets a sky-blue background to draw shapes on
```

Note you can create multiple turtles, which will be helpful for drawing different elements. See Using multiple turtles in the course notes.

```
####################################
# Create the turtle we will throw:
####################################
baby_turtle = turtle.Turtle()
baby_turtle.shape("turtle")

# Customize these colors yourself!
baby_turtle.color("orange", "green")

# You can use these defaults
baby_turtle.shapesize(2.5, 2.5, 3)
baby_turtle.pensize(3)

####################################
# Create a different turtle for writing text:
####################################
```

```
text_turtle = turtle.Turtle()
font = ('Arial', 20, 'bold')
text_turtle.hideturtle()
```

Finally, your program should end with the following code, to ensure the turtle program remains open until closed. See Keeping the Window Open in the course notes.

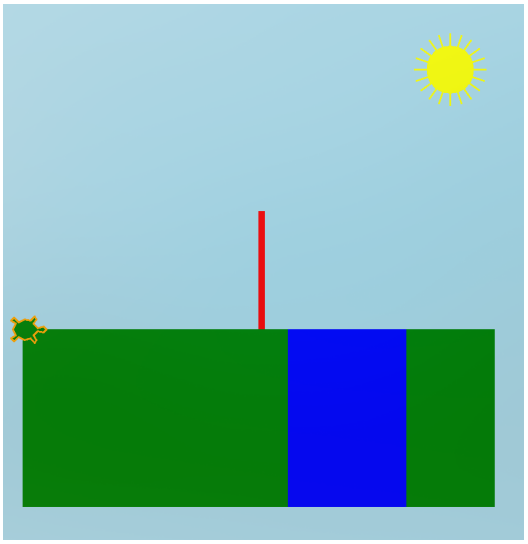
```
# IMPORTANT: don't have any code below this line, it won't run.
turtle.exitonclick()
```

1. Draw the Background - 20%

Your simulation requires:

- **ground**, and **ocean**, and a **wall**.
- **at least one extra unique element.** In the example below, I have drawn a sun in the top right corner – add your own creation to your game background.

Your background should look something like the figure below:



Example background for the turtle projectile simulation. The positions for where the ground, ocean, and walls should begin/end are defined in the Constants section of this assignment. Note that only one turtle icon is visible -- you should hide the turtle used to draw the background!

To create this background:

- See Colours and Backgrounds in the course notes for instructions on using the

turtle library.
- Use the constants defined below for the dimensions of these elements.

2. Initialization - 10%

The game should always begin with the turtle in the same starting position. Then, the game should ask whoever is running the program ("the user") for the initial velocity and the throwing angle. That is:

1. Set the turtle on the ground at position `(X_START, Y_START)`.
    - See the constants section below for suggested values.
2. Set your turtle to be facing the direction that they are moving
    - i.e. east, see Turn the turtle in the course notes.
3. Ask the user for:
    - the magnitude of the velocity vector (v) in (m/s)
    - angle from ground (θ) in degrees (integer)
    - See the Working with User Input part of the turtle course notes.
4. Create variables to store the the velocity vector's two initial components (v_x, v_y) based on the velocity magnitude (v) and angle (θ)
    - See the Math Module course notes.
    - Remember that you are entering angles in degrees, but the `math` module uses radians by default. Do not forget to convert as needed.

Simulate Basic Motion - 35%

Now it's time to throw the turtle! Remember the equations of motion defined in the Theory section of the beginning of the document? Now is the time to implement them in python.

That means:

- Convert the equations of motion to python statements
- Repeat these calculations over and over again to simulate the motion of the turtle.

We will use a For loop to repeat the calculations. Each time the loop repeats, the following should happen:

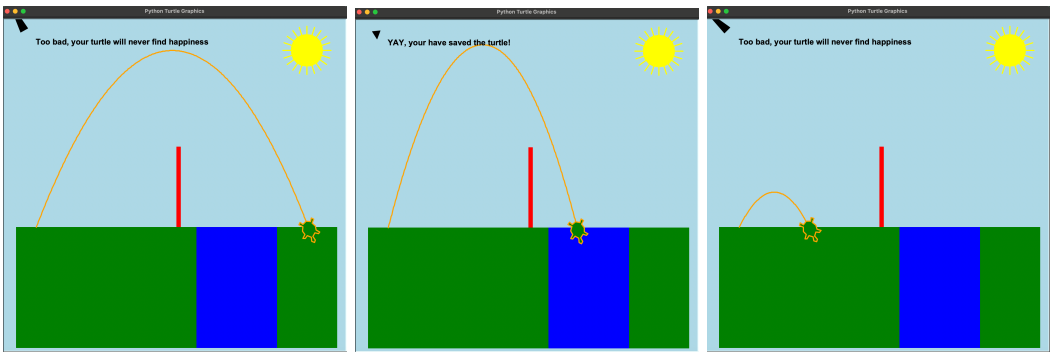
- Increment the values of the x and y positions, and the new vertical velocity v_y.
- At each time increment, v_y changes v_x – recalculate the angle θ that the turtle is

now headed towards.
    - Adjust the turtle angle so that the turtle is facing the direction that they are moving!

Experiment with different Δt values – the smaller, the more calculations will take place. (recommended: $\Delta t = 0.2$)

Stopping the Turtle when it hits the ground or the ocean - 20%

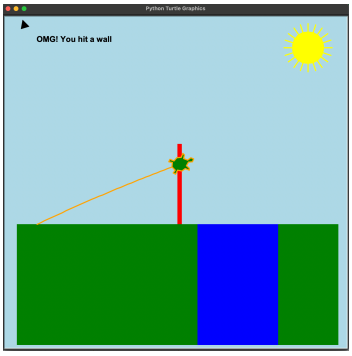
- Has the turtle hit the ground?
    - Print "**Too bad: your turtle is sad.**"
    - Stop looping. (via a `break` statement)
- Has the turtle landed in the ocean?
    - print "**Yay: you have saved the day.**"
    - Stop looping. (via a `break` statement)



These images show some of the possible fates that may befall your turtle.

Stopping the Turtle if it hits the wall - 10%

- Has the turtle hit the wall?
    - Print "**Pride goeth before a fall: another turtle has hit the wall**"
    - Stop looping. (via a `break` statement)



Wind - 5%

Add random wind to the equation, changing the velocity of the turtle at each Δt. It is up to you to determine how to modify the kinematic equations to account for the wind.

To get a random number, use the `random` library:

```
import random # put this line at the top of your program with the other import sta

wind_speed = random.randint(1, 20)  # random integer between 1 and 20
wind_direction = random.randint(0, 360) # random integer between 0 and 360
```

Constants

- Dimensions of turtle land

```
# turtle land dimensions
(X_MIN, Y_MIN, X_MAX, Y_MAX) = -400, -400, 400, 400
GROUND_LEVEL = -100  # the y value of the ground level
OCEAN_X1 = 50
OCEAN_Y1 = Y_MIN
OCEAN_X2 = 250
OCEAN_Y2 = GROUND_LEVEL
```

- The turtle starts at

```
X_START: int = X_MIN+50
Y_START: int = GROUND_LEVEL
```

- There is a retaining wall with the following dimensions:

```
# wall position
X_WALL_MIN = 0
X_WALL_MAX = 10
Y_WALL_MIN = GROUND_LEVEL
Y_WALL_MAX = 100
```

- Other constants you will need:

```
LOOP_CONSTANT = 1000000
```

Rubric

You will get grades for each of the above sections being complete and working correctly.

You will lose grades if…

- you don’t use proper variable names with meaning
- you hard code values instead of using variable names